Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

A implementação das funções de sincronização do embarque da METROREC em C com Pthreads (station_init, station_load_car, station_wait_for_car e station_on_board), em que cada vagão e cada passageiro é uma thread, usando apenas mutex e variáveis de condição, com um único lock por estação, embarque simultâneo e sem busy-wait.

Nesta entrega, todos os requisitos estão implementados e testados: o runner do professor passa (Looks good!) e passou em 50 execuções seguidas, e dois testes auxiliares comprovam a ausência de busy-wait e o comportamento em um caso limite e um caso inválido. Não ficou nenhum item sem implementar.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
struct station + station_init	OK	sync_car.h e station_init; runner começa sem erro
station_load_car só retorna com todos sentados e vagão cheio ou ninguém esperando	OK	./sync_car sem "returned early!"; termina em Looks good!
station_wait_for_car só retorna com vagão na estação e assento livre	OK	./sync_car sem "Many passengers on this car"; ./test_limites: caso inválido, 0 embarcaram
station_on_board avisa o vagão	OK	./test_limites: vagão só parte depois dos 3 on_board
embarque simultâneo	OK	runner chama vários wait_for_car antes dos on_board e passa
compilação por Makefile gerando sync_car	OK	make clean e make geram sync_car; segundo make diz "up to date"

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **sync_car.h → struct station (um mutex, três contadores - waiting_passengers, free_seats e boarding_passengers - e duas variáveis de condição - car_arrived e all_boarded) e os protótipos das quatro funções.**

2. **sync_car.c → station_init, station_wait_for_car, station_on_board e station_load_car; a estação funciona como um monitor**

3. **Makefile → compila sync_car.c + sync_car-runner.c em sync_car, com -g -pthread; alvo clean.**

Decisões técnicas:

1. **Contador boarding_passengers separado de waiting_passengers → Motivo: o enunciado distingue "reservou assento" (wait_for_car retornou) de "sentou" (on_board) → Consequência: vários passageiros embarcam ao mesmo tempo e o vagão nunca parte com alguém na porta.**
2. **pthread_cond_broadcast em load_car e pthread_cond_signal em on_board (só quando boarding_passengers chega a 0) → Motivo: um vagão com vários assentos deve liberar vários passageiros, mas só existe um vagão esperando por vez → Consequência: embarque simultâneo sem acordar o vagão à toa.**
3. **free_seats = 0 quando o vagão parte → Motivo: representar "não há vagão na estação" → Consequência: um passageiro que chega depois espera o próximo vagão em vez de embarcar num que já saiu.**
4. **struct station no sync_car.h, e não no sync_car.c como sugere o enunciado → Motivo: o runner declara struct station station; na pilha e precisa saber o tamanho da struct → Consequência: o runner compila sem nenhuma alteração.**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Monitor com dois mutexes	Um lock para os passageiros (passengers_lock), outro para o embarque (boarding_lock) e uma terceira condição (seat_taken) sinalizada no wait_for_car, achando que separar as partes deixaria o código mais organizado	Troquei de S1 para S2 ao perceber que dois locks violam o requisito 3; além disso, a decisão de partida do vagão lia contadores protegidos por locks diferentes (não atômica) e pegar um lock dentro do outro abria risco de deadlock
S2	Monitor com um mutex e duas condições	Um único lock; car_arrived para os passageiros (com broadcast) e all_boarded para o vagão (com signal quando boarding_passengers chega a 0); implementação função por função, compilando a cada passo	

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Rodar o runner com as funções vazias	Falha: station_load_car returned early!	load_car vazio retorna na hora, antes de os passageiros embarcarem; é o	02/11 ~19:02	Evidências.log, linhas 93–95

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
				ponto de partida do que o runner cobra		
2	S1	station_init e wait_for_car com dois mutexes e três condições	Compilou, mas a abordagem era inválida	Viola o requisito 3; a decisão de partida deixaria de ser atômica e o lock aninhado traz risco de deadlock	02/11 20:01	Evidências.log, linhas 192–193
3	S2	Voltar a um único lock e remover seat_taken; wait_for_car com while (free_seats == 0) cond_wait	Compilou		02/11 20:56	Evidências.log, linhas 197–205
4	S2	station_load_car com broadcast e espera pela regra de partida	Sucesso: Looks good!		02/11 23:01	Evidências.log, linhas 213–232
5	S2	Teste de ausência de busy-wait (10 passageiros sem vagão por 3 s)	Sucesso: 0,0056 s de CPU		03/11 18:36	Evidências.log, linhas 246–250
6	S2	Caso inválido (count = -5) e caso limite (10 assentos, 3 passageiros)	Sucesso		03/11 19:01	Evidências.log, linhas 259–263

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

1- Runner com as funções vazias, "Error: station_load_car returned early!".

```
Error: station_load_car returned early!
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ echo "sync_car" >> .gitignore
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ git add .
warning: in the working copy of '.gitignore', CRLF will be replaced by LF the next time Git touches i
t
warning: in the working copy of 'Makefile', CRLF will be replaced by LF the next time Git touches it
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$
```

2- Validação final: ./sync_car terminando em Looks good! e o laço de 50 execuções sem FALHOU.

```
Problems 1  Output  Debug Console  Terminal  Ports

lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ make
gcc -g -pthread sync_car.c sync_car-runner.c -o sync_car
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ ./sync_car
car entering station with 33 free seats
car departed station with 33 new passenger(s) (expected 33)
car entering station with 23 free seats
car departed station with 23 new passenger(s) (expected 23)
car entering station with 34 free seats
car departed station with 34 new passenger(s) (expected 34)
car entering station with 10 free seats
car departed station with 10 new passenger(s) (expected 10)
Looks good!
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ for i in $(seq 1 50); do ./sync_car > /dev/n
ull || echo "FALHOU na execucao $i"; done; echo "fim"
fim
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$
```

3- Ausência de busy-wait: ./test_busywait com "Tempo de CPU em 3 s de espera: 0.0056 s.

```
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ date; whoami; pwd
Tue Nov  3 18:34:45 -03 2026
lfvm
/mnt/c/Users/luisa/sync-car-metrorec
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ date
Tue Nov  3 18:36:41 -03 2026
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ gcc -pthread test_busywait.c sync_car.c -o t
est_busywait
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ ./test_busywait
Tempo de CPU em 3 s de espera: 0.0056 s
OK: sem busy-wait
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$
```

4- Casos limite e inválido: saída do ./test_limites.

```
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ gcc -pthread test_busywait.c sync_car.c -o t
est_busywait
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ ./test_busywait
Tempo de CPU em 3 s de espera: 0.0056 s
OK: sem busy-wait
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ echo "test_busywait" >> .gitignore
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$ date
gcc -pthread test_limites.c sync_car.c -o test_limites
./test_limites
echo "test_limites" >> .gitignore
Tue Nov  3 19:01:48 -03 2026
Caso invalido (count = -5): vagao retornou; embarcaram 0 (esperado 0)
Caso limite (10 assentos, 3 passageiros): embarcaram 3 (esperado 3)
free_seats depois da partida: 0 (esperado 0)
OK: casos limite e invalido passaram
lfvm@Lusa-Notebook:/mnt/c/Users/luisa/sync-car-metrorec$
```

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Usei o Claude no planejamento e para entender a teoria. Escrevi as quatro funções e o Makefile, e o Claude revisou cada uma, me alertando do erro quando usei dois locks. Ele também escreveu os testes auxiliares e ajudou a estruturar o relatório.

Prompts principais (liste, sem história):

"Como testo a parte que fiz?"

"Antes de eu dar qualquer commit, que coisas eu posso ter errado aqui que darão erros lá na frente?"

"Faz um documento unindo o que eu implementei com o conteúdo do livro, explicando como tudo se conecta, já esperando possíveis perguntas quando eu for apresentar"

O que validei manualmente e como:

Compilei e rodei tudo no meu terminal, com a sessão gravada no evidencias.log. Conferi que o runner termina em Looks good! e rodei o laço de 50 execuções sem nenhuma falha. Rodei os dois testes auxiliares e conferi as saídas: 0,0056 s de CPU com 10 threads esperando e os casos limite e inválido com o número esperado de passageiros. Conferi cada requisito do enunciado contra o meu código (seção 2) e entendi por que a versão com dois locks estava errada antes de trocá-la.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Diferente de outras entregas, desta vez a gestão de tempo funcionou: planejei (no final de semana) a segunda para o código e a terça para testes e relatório. O principal erro foi de desenho: comecei com dois mutexes porque separar "passageiros" de "embarque" parecia mais organizado, sem reler as restrições do enunciado, e isso violava o requisito 3 e deixava a decisão de partida do vagão sujeita a condição de corrida. Aprendi que, em concorrência, a pergunta certa é "quem dorme esperando o quê, e quem acorda quem", e que passar uma vez no teste não prova nada, por isso repeti a execução 50 vezes. Com mais tempo, teria escrito a regra de partida do vagão como expressão lógica antes de começar a codar.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Priorizaria três coisas. Tratamento de erros: hoje ignoro o retorno de pthread_mutex_init, pthread_cond_init, lock e wait, checaria cada um e mostraria uma mensagem clara em caso de falha, no mesmo formato em todo o código. Legibilidade: a condição de espera do vagão (boarding_passengers > 0 || (free_seats > 0 && waiting_passengers > 0)) é difícil de ler de primeira, eu a colocaria numa função auxiliar com um nome claro, como pode_partir, e comentaria em cada função o que cada contador representa e quem acorda quem. Organização do repositório: faria todos os commits pelo terminal gravado, para o histórico do Git e o evidencias.log contarem a mesma história, e deixaria o make clean apagando também os executáveis dos testes auxiliares.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.